

---

# CS121 Data Structures in C

## 04 *Linked Lists*

# Outline

---

- Linked list
- Implementation in C
  - ♦ Insert an item in the list
  - ♦ Remove an item from the list
- Common errors in the implementation
- Doubly linked list
- Implementation in C
  - ♦ Insert an item in the list
  - ♦ Remove an item from the list

# Linked Lists (1)

---

- Disadvantages of array implementation of lists
  - ♦ The list size must be known in advance.
  - ♦ Insertion and deletion operations are expensive.

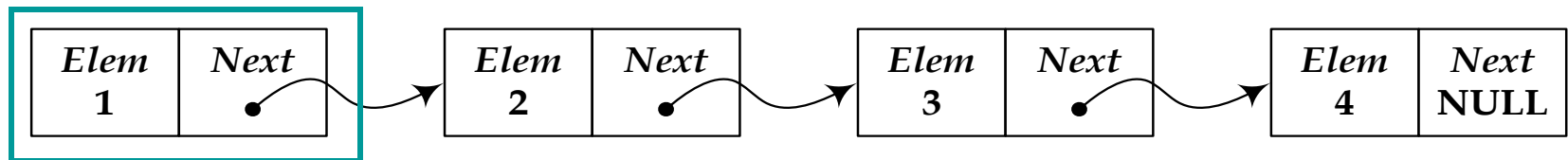
# Linked Lists (2)

- Singly-linked lists (simply called linked lists)

A series of connected nodes. Each node contains at least:

Element: the data/the pure item;

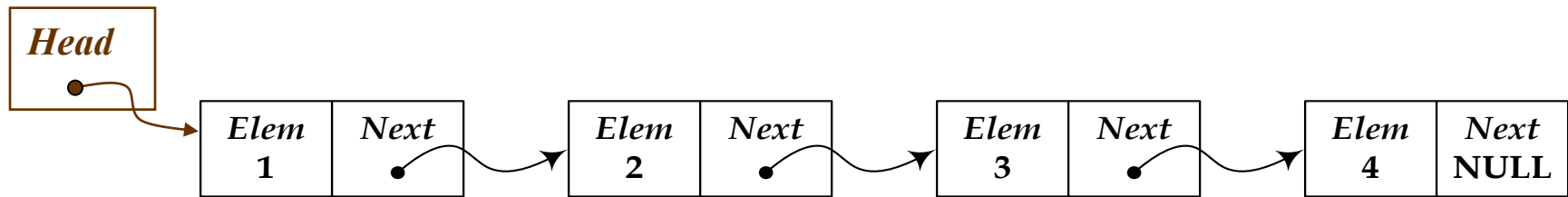
Pointer: points to its successor (its next node), called the **Next** pointer.



Node: the element and the pointer.

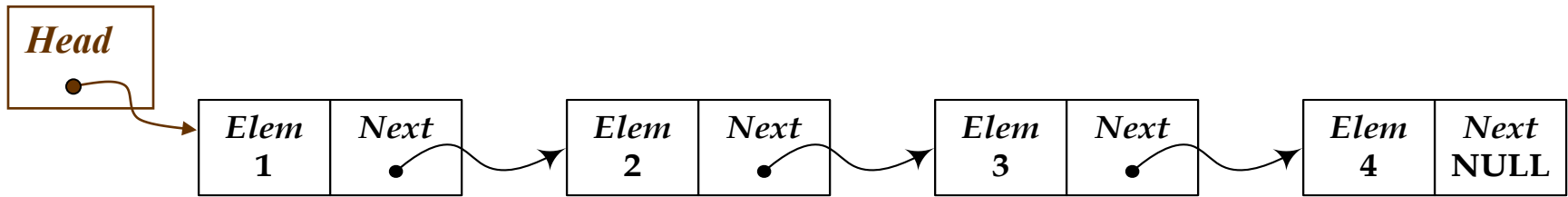
# Implementation Linked Lists in C (1)

- In practice, the start of the list is the head node. (Our convention: the head is in position 0.)



- Why?
  - ◆ Make sure the position of the list in memory;
  - ◆ Don't lose the list when deleting from the front of the list;

# Implementation Linked Lists in C (2)



- The linked list ADT

- ♦ The structure of a node:

```
typedef int ElementType;           /* list.h */
typedef struct Node *PtrToNode;
typedef PtrToNode Position;
```

```
struct Node                        /* list.c */
{
    ElementType    Element;
    PtrToNode      Next;
};
```

# Implementation Linked Lists in C (3)

- Space allocation
  - ◆ Linked list is a dynamic data structure.
  - ◆ Nodes are dynamically allocated. Dynamic data are usually allocated on the C runtime heap.
    - Every node needs space.
    - Lifecycle and scope.

```
PtrToNode P = malloc(sizeof(struct Node));  
P->Elem = ...  
P->Next = ...
```

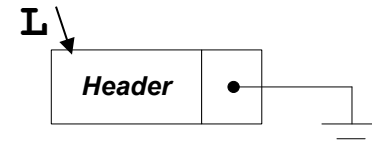
# Implementation Linked Lists in C (4)

- The function/operation prototypes (list.h)

```
List MakeEmpty( List L );  
int IsEmpty( List L );  
int IsLast( Position P, List L );  
Position Find( ElementType X, List L );  
void Delete( ElementType X, List L );  
Position FindPrevious( ElementType X, List L );  
void Insert( ElementType X, List L, Position P );  
void DeleteList( List L );  
Position Header( List L );  
Position First( List L );  
Position Advance( Position P ); /* Return the next of P node */  
ElementType Retrieve( Position P );  
/* Get the data of P node*/
```

# Implementation Linked Lists in C (5)

- The empty list **L** with header



```
/* Return true if L is empty */
```

```
int IsEmpty( List L )
```

```
{
```

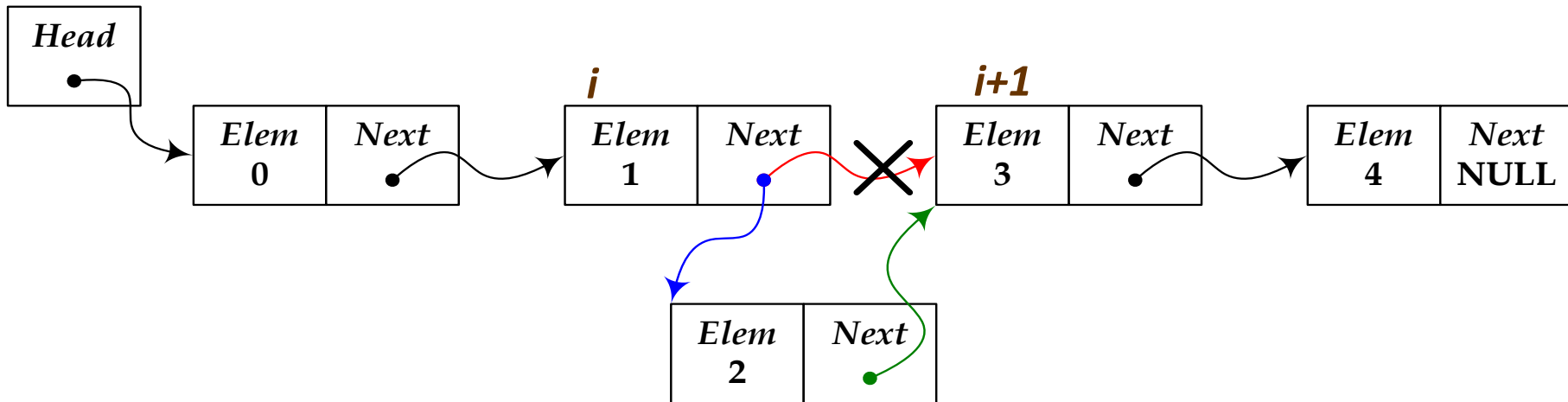
```
    return L->Next == NULL;
```

```
}
```

A pointer to a list

# Operations -- Insert An Item (1)

- To insert an item into the list after position  $i$



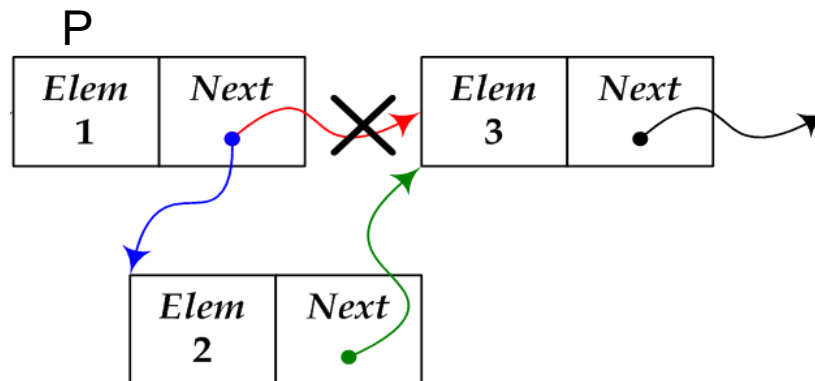
- ◆ Chain the new node to its successor:  
Set the **Next pointer** of the new item to point to the original  $i+1$  item;
- ◆ Chain the predecessor to the new node:  
Set the **Next pointer** of the  $i$  item to point to the new item.

# Operations -- Insert An Item (2)

```
void Insert( ElementType X, List L, Position P )
```

- Steps

1. Allocate memory for the new node
2. Point the new node to P's successor
3. Point P's next node to the new node



# Operations -- Insert An Item (3)

- To insert an item into the list after position  $i$

```
void Insert( ElementType X, List L, Position P )
{
    Position TmpCell;

    /*1*/    TmpCell = malloc( sizeof( struct Node ) );
    /*2*/    if( TmpCell == NULL )
    /*3*/        FatalError( "Out of space!!!" );

    /*4*/    TmpCell->Element = X;
    /*5*/    TmpCell->Next = P->Next;

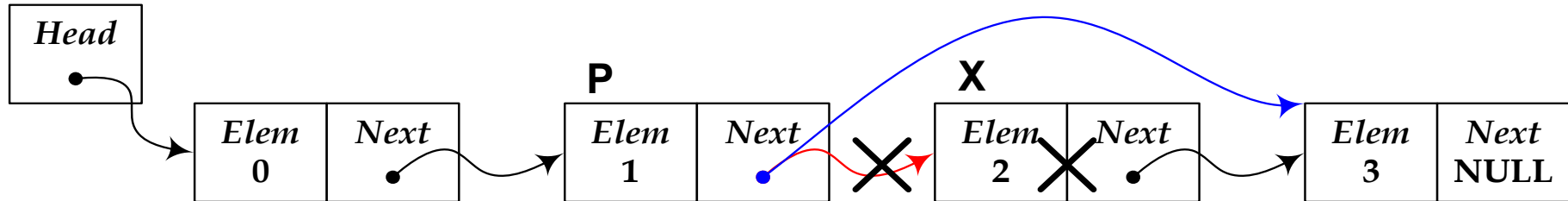
    /*6*/    P->Next = TmpCell;
}
```

# Operations -- Insert An Item (4)

- More consideration:
  - ♦ If there is a header, insert the new node at  $i = 0$  is the position of *Head- $\rightarrow$ Next*.
  - ♦ If we reach NULL *BEFORE* Node ( $i-1$ ), then return ERROR. (over range)
- Time complexity:  $O(1)$  for the algorithm in textbook (in general, it is  $O(N)$  for *Insert*)
  - ♦ In all cases, only a fixed number of instructions are performed, no matter how large the list is.

# Operations -- Remove An Item (1)

- To delete the node X from the list L (the first occurrence (出現) of X and does nothing if X is not in the list)



- ◆ First, find the node P which is the node prior to the one containing X.
- ◆ Then, chain the X node's predecessor to the X node's successor.
- ◆ Last, free the space allocated for the node (to delete it from the memory).

# Operations -- Remove An Item (2)

- To delete the node X after the legal position P from the list L

```
void Delete( ElementType X, List L )
{
    Position P, TmpCell;
    // if X is found, to return the predecessor node P
    // if X isn't found, to return the last node
    P = FindPrevious( X, L );

    if( !IsLast( P, L ) )    /* P isn't the last node in L */
    {

        TmpCell = P->Next;

        P->Next = TmpCell->Next;

        free( TmpCell );
    }
}
```

# Operations -- Remove An Item (3)

- Find the predecessor P of X

```
Position FindPrevious( ElementType X, List L )
{
    Position P;

    /*1*/    P = L; //start from the header
    /*2*/    while( P->Next != NULL && P->Next->Element != X )
    /*3*/        P = P->Next; //move to the next node

    /*4*/    return P;
}
```

- Time complexity:  $O(N)$  in the worst case, because the entire list might need to be traversed.  $N$  is the number of the list.

# Most Efficient Operations on Linked Lists

- Inserting after the head position:

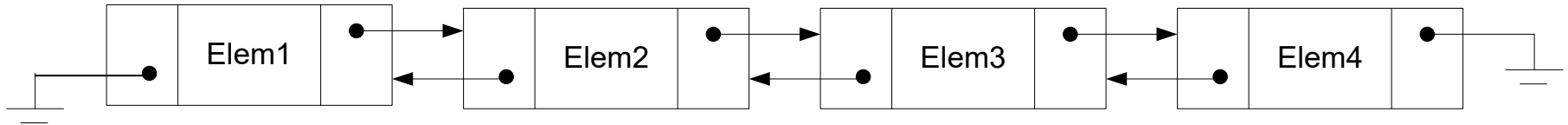
```
Position TmpCell = malloc( sizeof( struct Node ) )  
...  
TmpCell->Next = L->Next;  
L->Next = TmpCell;
```

- Removing the first node after the head node:

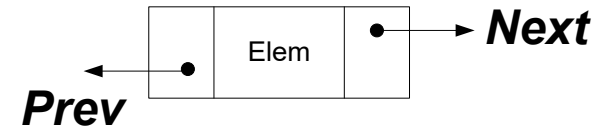
```
if ( L->Next ) {  
    Position TmpCell = L->Next;  
    L->Next = TmpCell->Next;  
    free(TmpCell);  
}
```

- Time complexity?
  - ♦ Constant time:  $O(1)$

# Doubly Linked Lists



- Doubly linked list: adding a *Prev* pointer to a node, and let it point to the previous node, so it can traverse two directions
- Each node:
  - ◆ a data member,
  - ◆ a pointer to the next element
  - ◆ a pointer to the previous element
- To mark the head and tail of the list
  - ◆ set the *Prev* pointer of the first element and the *Next* pointer of the last element to **NULL**.



# Implementation Dlinked Lists in C (1)

- Define a structure for doubly-linked list node

```
struct DNode;  
typedef struct DNode  *PtrToDNode;  
typedef PtrToDNode    DList;  
  
struct DNode  
{  
    ElementType  Element;  
    struct DNode *Prev;  
    struct DNode *Next;  
};
```

# Implementation Dlinked List in C (2)

- The functions/ operations
  - ♦ The same as linked list if it is only related with one directional pointers, such as:

```
Position Find( ElementType X, DList L );  
Position FindPrevious( ElementType X, DList L );  
ElementType Retrieve( Position P );  
.....
```

- ♦ The difference in handling Delete and Insert operations, which are related with two directional pointers.

```
void Insert( ElementType X, DList L, Position P );  
void Delete( ElementType X, DList L );
```

- ♦ Time complexity:  $O(N)$

# Operations – Insert An Item (1)

- To insert an element X after the position P (Only considering the simplest case for clarity: the list is not empty. )

- ♦ Steps:

Create the new node;

Manage the pointer connection in two directions: the primary difference from the insertion algorithm in the single linked list

```
typedef int      ElementType;  
typedef struct DNode *PtrToDNode;  
typedef PtrToDNode DList;  
typedef PtrToDNode Position;
```

# Operations – Insert An Item (2)

//Only considering the simplest case for clarity: the list is not empty.

```
void Insert_DList( ElementType X, DList L, Position P )
{
    Position TmpCell;
    //Create a new node, and always check if the allocation is
    //successful.
    /*1*/ TmpCell = malloc( sizeof( struct DNode ) );
    /*2*/ if( TmpCell == NULL )
    /*3*/     FatalError( "Out of space!!!" );

    /*4*/ TmpCell->Element = X;

    ① TmpCell->Prev = P;
    ② TmpCell->Next = P->Next;
    ③ P->Next->Prev = TmpCell;
    ④ P->Next = TmpCell;
}
```

# Operations – Remove An Item (1)

- To delete first occurrence of X from the list (Only considering the simplest case for clarity: the list is not empty and  $0 < \text{Position}(X) < \text{Length}(\text{List})$  )
  - ♦ Steps:
    - Find the predecessor of the first occurrence of X;
    - Manage the pointer connection in two directions: the primary difference from the deletion algorithm in the single linked list

# Operations – Remove An Item (2)

```
void Delete_DList( ElementType X, DList L )
{
    Position P, TmpCell;
    // if X is found, to return the predecessor node P
    P = FindPrevious( X, L );

    if( !IsLast( P, L ) )    /* P isn't the last node in L */
    {
        //record the address of node X
        TmpCell = P->Next;

        //chain the predecessor to the successor of X in the
        //forward direction
        P->Next = TmpCell->Next;
        //chain the successor to the predecessor of X in the
        //backward direction
        P->Next->Prev = P;
        //release the memory
        free( TmpCell );
    }
}
```

the simplest case for clarity: the list is not empty and  $0 < \text{Position}(X) < \text{Length}(\text{List})$

# Conclusion (1)

---

- List is one of the most basic data structures
- Two implementations
  - ◆ Array (static)
  - ◆ Linked list (dynamic)
- Main operations
  - ◆ Insert, Delete, Find...
  - ◆ The time complexity
  - ◆ Handle carefully about the special cases
    - When the list is full or empty

# Conclusion (2)

---

- Array versus Linked Lists

Linked lists are more complex to code and manage than arrays, but they have some distinct advantages.

- ♦ Dynamic: a linked list can easily grow and shrink in size.

- We don't need to know how many nodes will be in the list. They are created in memory as needed.
- In contrast, the size of an array is fixed at compilation time.

- ♦ Easy and fast insertions and deletions

- To insert or delete an element in an array, we need to make room in the array for new elements or close the gap caused by deleted elements.
- With a linked list, no need to move other nodes. Only need to reset some pointers.

# Assignment 03

---

- 3.1 - 3.3
- Submit it via moodle.